



[Latest](#)
[Courses »](#)
[Advice](#)
[Bringers](#)
[Everything Else »](#)

Android, MySQL, PHP, & JSON 6 | JSON Parsing & Android ListView Design

[Home](#)

[Tutorial Series](#)

Android, MySQL, PHP, & JSON 6 | JSON Parsing & Android ListView Design

Posted By trav on Jun 10, 2013 | 0 comments

This entry is part 22 of 22 in the series [Android-Intermediate](#)

Parsing JSON Data into an Android ListView!



In this Android tutorial we'll be parsing JSON data displaying the results nicely in a list view. We will learn how to stylize our list to make it visually appealing. We will also create our post comment activity, which won't be much different than the previous tutorial, and we will also create our read comments activity.

Follow the Remote Databases for Android Series!

- [Part 1: Android Remote Databases and Servers Overview](#)
- [Part 2: Local Xampp Server and MySQL Database](#)
- [Part 3: Working with a Remote Server and MySQL](#)
- [Part 4: Creating Login and Registration PHP Scripts](#)
- [Part 5: Developing the Android Application](#)
- **Part 6: JSON Parsing and Android ListView Design**



Remote

Databases Part 6 – ListView Design Video Tutorial!

Coming soon...

Finishing Up the Android Remote Databases Mini Series

Our mini series is coming to a swift end. All of our hard work will be polished with some old-school mybringback



swagger. All we need to do is take advantage of our other PHP scripts which allow us to add comments, and also allows us to read through the comments posted by all users. Lastly, we will add some stylization to our Android ListView that will display all of our comments.

Step 1: Creating a Post Comment Activity

This step will be pretty similar to the login and register activities. We will just have to post our username, title, and message to our **"addcomment.php"** script. I won't go into much detail since this is pretty similar to the last tutorial.

add_comment.xml

```

1  <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
2      xmlns:tools="http://schemas.android.com/tools"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      >
6
7      <ImageView
8          android:id="@+id/imageView1"
9          android:layout_width="wrap_content"
10         android:layout_height="wrap_content"
11         android:layout_below="@+id/textView2"
12         android:layout_centerHorizontal="true"
13         android:src="@drawable/arrowstars" />
14
15     <TextView
16         android:id="@+id/textView2"
17         android:layout_width="wrap_content"
18         android:layout_height="wrap_content"
19         android:layout_alignParentLeft="true"
20         android:layout_alignParentTop="true"
21         android:layout_marginTop="16dp"
22         android:gravity="center"
23         android:text="Android Remote Server Tutorial"
24         android:textAppearance="?android:attr/textAppearanceLarge"
25         android:textStyle="bold" />
26
27     <EditText
28         android:id="@+id/message"
29         android:layout_width="wrap_content"
30         android:layout_height="wrap_content"
31         android:layout_above="@+id/submit"
32         android:layout_alignParentLeft="true"
33         android:layout_alignRight="@+id/textView2"
34         android:layout_below="@+id/TextView01"
35         android:ems="10"
36         />
37
38     <EditText
39         android:id="@+id/title"
40         android:layout_width="wrap_content"
41         android:layout_height="wrap_content"
42         android:layout_alignParentLeft="true"
43         android:layout_alignParentRight="true"
44         android:layout_below="@+id/textView1"
45         android:ems="10" >
46
47         <requestFocus />
48     </EditText>
49
50     <TextView
51         android:id="@+id/textView1"
52         android:layout_width="wrap_content"
53         android:layout_height="wrap_content"
54         android:layout_centerVertical="true"
55         android:layout_toLeftOf="@+id/imageView1"
56         android:text="Title" />
57
58     <Button
59         android:id="@+id/submit"
60         android:layout_width="wrap_content"
61         android:layout_height="wrap_content"
62         android:layout_alignParentBottom="true"
63         android:layout_alignParentLeft="true"
64         android:layout_alignParentRight="true"
65         android:text="Submit" />
66
67     <TextView
68         android:id="@+id/TextView01"
69         android:layout_width="wrap_content"
70         android:layout_height="wrap_content"
71         android:layout_alignLeft="@+id/textView1"
72         android:layout_below="@+id/title"
73         android:layout_marginTop="14dp"
74         android:text="Message" />
75

```

```
76 | </RelativeLayout>
```

Now, let's setup our add comment activity, this will be version one, because we will need a way to determine who the user is, and we will post the username on their behalf.

AddComment.java v0.1

```
1 | package com.example.mysqltest;
2 |
3 | import java.util.ArrayList;
4 | import java.util.List;
5 | import org.apache.http.NameValuePair;
6 | import org.apache.http.message.BasicNameValuePair;
7 | import org.json.JSONException;
8 | import org.json.JSONObject;
9 | import android.app.Activity;
10 | import android.app.ProgressDialog;
11 | import android.os.AsyncTask;
12 | import android.os.Bundle;
13 | import android.util.Log;
14 | import android.view.View;
15 | import android.view.View.OnClickListener;
16 | import android.widget.Button;
17 | import android.widget.EditText;
18 | import android.widget.Toast;
19 |
20 | public class AddComment extends Activity implements OnClickListener{
21 |
22 |     private EditText title, message;
23 |     private Button mSubmit;
24 |
25 |     // Progress Dialog
26 |     private ProgressDialog pDialog;
27 |
28 |     // JSON parser class
29 |     JSONObject jsonParser = new JSONObject();
30 |
31 |     //php login script
32 |
33 |     //localhost :
34 |     //testing on your device
35 |     //put your local ip instead, on windows, run CMD > ipconfig
36 |     //or in mac's terminal type ifconfig and look for the ip under en0 or en1
37 |     // private static final String POST_COMMENT_URL =
38 |     "http://xxx.xxx.x.x:1234/webservice/addcomment.php";
39 |
40 |     //testing on Emulator:
41 |     private static final String POST_COMMENT_URL =
42 |     "http://10.0.2.2:1234/webservice/addcomment.php";
43 |
44 |     //testing from a real server:
45 |     //private static final String POST_COMMENT_URL =
46 |     "http://www.mybringback.com/webservice/addcomment.php";
47 |
48 |     //ids
49 |     private static final String TAG_SUCCESS = "success";
50 |     private static final String TAG_MESSAGE = "message";
51 |
52 |     @Override
53 |     protected void onCreate(Bundle savedInstanceState) {
54 |         // TODO Auto-generated method stub
55 |         super.onCreate(savedInstanceState);
56 |         setContentView(R.layout.add_comment);
57 |
58 |         title = (EditText)findViewById(R.id.title);
59 |         message = (EditText)findViewById(R.id.message);
60 |
61 |         mSubmit = (Button)findViewById(R.id.submit);
62 |         mSubmit.setOnClickListener(this);
63 |
64 |     }
65 |
66 |     @Override
67 |     public void onClick(View v) {
68 |         new PostComment().execute();
69 |     }
70 |
71 |     class PostComment extends AsyncTask<String, String, String> {
72 |
73 |         @Override
74 |         protected void onPreExecute() {
75 |             super.onPreExecute();
76 |             pDialog = new ProgressDialog(AddComment.this);
77 |             pDialog.setMessage("Posting Comment...");
78 |             pDialog.setIndeterminate(false);
79 |             pDialog.setCancelable(true);
80 |             pDialog.show();
81 |         }
82 |     }
83 | }
```

```

81  @Override
82  protected String doInBackground(String... args) {
83      // TODO Auto-generated method stub
84      // Check for success tag
85      int success;

86      String post_title = title.getText().toString();
87      String post_message = message.getText().toString();
88
89      //We need to change this:
90      String post_username = "temp";
91
92      try {
93          // Building Parameters
94          List<NameValuePair> params = new ArrayList<NameValuePair>();
95          params.add(new BasicNameValuePair("username", post_username));
96          params.add(new BasicNameValuePair("title", post_title));
97          params.add(new BasicNameValuePair("message", post_message));
98
99          Log.d("request!", "starting");
100
101          //Posting user data to script
102          JSONObject json = jsonParser.makeHttpRequest(
103              POST_COMMENT_URL, "POST", params);
104
105          // full json response
106          Log.d("Post Comment attempt", json.toString());
107
108          // json success element
109          success = json.getInt(TAG_SUCCESS);
110          if (success == 1) {
111              Log.d("Comment Added!", json.toString());
112              finish();
113              return json.getString(TAG_MESSAGE);
114          } else {
115              Log.d("Comment Failure!", json.getString(TAG_MESSAGE));
116              return json.getString(TAG_MESSAGE);
117          }
118      } catch (JSONException e) {
119          e.printStackTrace();
120      }
121
122      return null;
123
124  }
125
126
127  protected void onPostExecute(String file_url) {
128      // dismiss the dialog once product deleted
129      pDialog.dismiss();
130      if (file_url != null){
131          Toast.makeText(AddComment.this, file_url, Toast.LENGTH_LONG).show();
132      }
133
134  }
135
136  }
137
138  }
139  }

```

Step 2: Saving User Data with Shared Preferences

Right now, we can use our add comment activity, but whatever we post will be posted as the username "temp" because we don't have nor do we want an input field for the username. Instead, when there is a successful login, we are going to save the username in SharedPreferences, and then we will retrieve that information when the user posts a comment. First, let's modify our "Login.java" file:

Login.java

```

1  ...
2  //upon successful login, save username:
3  // Async json success tag
4  success = json.getInt(TAG_SUCCESS);
5  if (success == 1) {
6      Log.d("Login Successful!", json.toString());
7      // save user data
8      SharedPreferences sp = PreferenceManager
9          .getDefaultSharedPreferences(Login.this);
10     Editor edit = sp.edit();
11     edit.putString("username", username);
12     edit.commit();
13
14     Intent i = new Intent(Login.this, ReadComments.class);
15     finish();
16     startActivity(i);
17     return json.getString(TAG_MESSAGE);
18 } else {
19     Log.d("Login Failure!", json.getString(TAG_MESSAGE));

```

```

20         return json.getString(TAG_MESSAGE);
21     }
22
23     ...

```

Now we have saved the username with SharedPreferences when there is a successful login. We save it as a key-value pair with the key being "username" and the value is the string data from the username EditText when the user last tried to log in. All we have to do now is modify our **AddComment.java** file to retrieve the saved data from our shared preferences with the key of "username".

AddComment.java

```

1  ...
2      String post_title = title.getText().toString();
3      String post_message = message.getText().toString();
4
5      //Retrieving Saved Username Data:
6      SharedPreferences sp =
7      PreferenceManager.getDefaultSharedPreferences(AddComment.this);
8      String post_username = sp.getString("username", "anon");
9
10     try {
11         // Building Parameters

```

Step 3: Creating a XML Template for JSON Elements!

Cool, now we should have our commenting system setup and working properly. However, we have no way of accessing our "AddComment" activity, so there is no way to test it. You'll just have to take my word that it works. Especially, because if it doesn't I'll come back to the previous section and edit it before you even know I originally messed up... unless I'm too lazy to test it out later or forget to edit my errors. If that's the case, tough luck kid.

Anyway, let's setup the last piece of our Android remote database series, parsing through all of the current comments in the database and displaying them in a ListView. Before, you test this out, you may want to add some comments to our comments table in our MySQL database, but I'll leave that up to you.

First, let's look at our JSON data when we go to our **comments.php** script. It should look something like this (once we format it):

```

1  { "message" : "Post Available!",
2    "posts" : [ { "message" : "Here is my message",
3                  "post_id" : "1",
4                  "title" : "Testing this title",
5                  "username" : "trav"
6                },
7                { "message" : "This is going to be awesome!",
8                  "post_id" : "2",
9                  "title" : "Title 2",
10                 "username" : "trav"
11               },
12               { "message" : "... like really bored.",
13                 "post_id" : "3",
14                 "title" : "I'm really bored",
15                 "username" : "bored guy"
16               },
17               { "message" : "This isn't roger, it's still travis",
18                 "post_id" : "4",
19                 "title" : "Just kidding",
20                 "username" : "roger"
21               },
22               { "message" : "tjw",
23                 "post_id" : "5",
24                 "title" : "test",
25                 "username" : "temp"
26               },
27               { "message" : "did it work",
28                 "post_id" : "6",
29                 "title" : "testing shared prefs",
30                 "username" : "anon"
31               }
32             ],
33   "success" : 1
34 }

```

The first thing we going to look for is the success message. If a 1 is returned, then we can continue to parse the rest of the JSON data. We will want to extract the username, the title, and the message. Once we get all of this information we want to put it into a list item and display all of our list items in a list view. We can use the post_id to tell which "posts" are the most recent. The higher the post_id, the more recent the post. For our application we will probably just list the 15 most recent post for this example.

If you want to continue building upon this project, you may want to write some more PHP scripts that return the JSON data of posts between two dates or something. (You would have to add a data field set into the "comments"

table of our MySQL database and make the adjustments to the "posting a comment" script so that it will POST a the current date.)

The first thing that we will have to do is develop a xml template for a single list item.

single_post.xml

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="fill_parent"
4      android:layout_height="wrap_content"
5      android:background="#f0f0f0"
6      android:orientation="vertical" >
7
8      <LinearLayout
9          android:layout_width="match_parent"
10         android:layout_height="wrap_content"
11         android:layout_margin="5dp"
12         android:background="@drawable/post_border_style"
13         android:orientation="vertical" >
14
15         <LinearLayout
16             android:id="@+id/box"
17             android:layout_width="match_parent"
18             android:layout_height="wrap_content"
19             android:layout_margin="2dp"
20             android:background="@drawable/post_background_style"
21             android:orientation="horizontal" >
22
23             <ImageView
24                 android:id="@+id/imageView1"
25                 android:layout_width="wrap_content"
26                 android:layout_height="wrap_content"
27                 android:src="@drawable/arrowstars" />
28
29             <LinearLayout
30                 android:id="@+id/box"
31                 android:layout_width="match_parent"
32                 android:layout_height="wrap_content"
33                 android:layout_margin="2dp"
34                 android:orientation="vertical"
35                 android:padding="5dp" >
36
37                 <TextView
38                     android:id="@+id/title"
39                     android:layout_width="wrap_content"
40                     android:layout_height="wrap_content"
41                     android:layout_gravity="bottom"
42                     android:paddingBottom="2dip"
43                     android:paddingLeft="5dp"
44                     android:paddingTop="6dip"
45                     android:textColor="#333"
46                     android:textSize="16sp"
47                     android:textStyle="bold" />
48
49                 <TextView
50                     android:id="@+id/message"
51                     android:layout_width="wrap_content"
52                     android:layout_height="wrap_content"
53                     android:paddingBottom="2dip"
54                     android:paddingLeft="8dp"
55                     android:textColor="#888" >
56                 </TextView>
57
58                 <LinearLayout
59                     android:layout_width="match_parent"
60                     android:layout_height="wrap_content"
61                     android:orientation="horizontal"
62                     android:paddingBottom="5dp" >
63
64                     <TextView
65                         android:layout_width="wrap_content"
66                         android:layout_height="wrap_content"
67                         android:gravity="left"
68                         android:paddingLeft="5dp"
69                         android:text="Username: "
70                         android:textColor="#5d5d5d"
71                         android:textStyle="bold" >
72                     </TextView>
73
74                     <TextView
75                         android:id="@+id/username"
76                         android:layout_width="wrap_content"
77                         android:layout_height="wrap_content"
78                         android:gravity="left"
79                         android:textColor="#acacac"
80                         android:textStyle="bold" >
81                     </TextView>
82                 </LinearLayout>
83             </LinearLayout>
84         </LinearLayout>
85     </LinearLayout>

```

86 | </LinearLayout>

Step 4: Adding Styles and Helper Files.

You have probably noticed that there are a lot of references to the drawable folder which acts as a background or a style of some sort. Let's setup those XML files. Right-click your "res" folder and add a new folder called "drawable". Right-click on the newly created drawable folder and add a "New.." > "Android XML File". Make sure the resource type is drawable, and create the following files:

- black_button.xml
- blue_gradient.xml
- post_background_style.xml
- post_border_style.xml

black_button.xml

```

1 | <?xml version="1.0" encoding="utf-8"?>
2 | <selector xmlns:android="http://schemas.android.com/apk/res/android">
3 |
4 |     <item android:state_pressed="true"><shape>
5 |         <solid android:color="#343434" />
6 |
7 |         <stroke android:width="1dp" android:color="#171717" />
8 |
9 |         <corners android:radius="3dp" />
10 |
11 |         <padding android:bottom="10dp" android:left="10dp" android:right="10dp"
12 | android:top="10dp" />
13 |     </shape></item>
14 |     <item><shape>
15 |         <gradient android:angle="270" android:endColor="#171717"
16 | android:startColor="#343434" />
17 |
18 |         <stroke android:width="1dp" android:color="#171717" />
19 |
20 |         <corners android:radius="4dp" />
21 |
22 |         <padding android:bottom="10dp" android:left="10dp" android:right="10dp"
23 | android:top="10dp" />
24 |     </shape></item>
25 | </selector>

```

blue_gradient.xml

```

1 | <?xml version="1.0" encoding="utf-8"?>
2 | <selector xmlns:android="http://schemas.android.com/apk/res/android">
3 |
4 |     <item android:state_pressed="true"><shape>
5 |         <solid android:color="#1bccf3" />
6 |
7 |         <stroke android:width="1dp" android:color="#1b60f3" />
8 |
9 |         <padding android:bottom="10dp" android:left="10dp" android:right="10dp"
10 | android:top="10dp" />
11 |     </shape></item>
12 |     <item><shape>
13 |         <gradient android:angle="270" android:endColor="#1b60f3"
14 | android:startColor="#1bccf3" />
15 |
16 |         <stroke android:width="1dp" android:color="#1b60f3" />
17 |
18 |         <padding android:bottom="10dp" android:left="10dp" android:right="10dp"
19 | android:top="10dp" />
20 |     </shape></item>
21 | </selector>

```

post_background_style.xml

```

1 | <?xml version="1.0" encoding="utf-8"?>
2 | <selector xmlns:android="http://schemas.android.com/apk/res/android">
3 |
4 |     <item android:state_pressed="true"><shape>
5 |         <solid android:color="#cccccc" />
6 |     </shape></item>
7 |     <item><shape>
8 |         <gradient android:angle="90" android:endColor="#f5f5f5"
9 | android:startColor="#fff" />
10 |     </shape></item>
11 | </selector>

```

post_border_style.xml


```

1 <?xml version="1.0" encoding="utf-8"?>
2 <selector xmlns:android="http://schemas.android.com/apk/res/android">
3
4     <item android:state_pressed="true"><shape>
5         <solid android:color="#cccccc" />
6     </shape></item>
7     <item><shape>
8         <gradient android:angle="270" android:endColor="#ccc"
9         android:startColor="#fff" />
10        <corners android:radius="2dp" />
11    </shape></item>
12
13 </selector>

```

Since, we are adding helper files, let go open up our values/**strings.xml** and modify it.

strings.xml

```

1 <resources>
2
3     <!-- string ids for registration.xml -->
4     <string name="app_name">Remote Databases</string>
5     <string name="username_textview">Username:</string>
6     <string name="password_textview">Password:</string>
7     <string name="submit">Submit</string>
8
9     <!-- string ids for home.xml -->
10    <string name="post_comment">Post Comment</string>
11    <string name="read_comments_title">Recent Comments</string>
12    <string name="passwords_not_same">Passwords don't match</string>
13 </resources>

```

Right-click on the "values" folder and add a "New.." > "Android XML File" of the resource type "value", and label this new value XML file **style.xml**.

styles.xml

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3
4     <!--
5         Base application theme, dependent on API level. This theme is replaced
6         by AppBaseTheme from res/values-vXX/styles.xml on newer devices.
7     -->
8     <style name="AppBaseTheme" parent="android:Theme.Light">
9         <!--
10            Theme customizations available in newer API levels can go in
11            res/values-vXX/styles.xml, while customizations related to
12            backward-compatibility can go here.
13        -->
14    </style>
15
16    <!-- Application theme. -->
17    <style name="AppTheme" parent="AppBaseTheme">
18        <!-- All customizations that are NOT specific to a particular API-level can
19        go here. -->
20    </style>
21
22
23    <style name="WhiteText">
24        <item name="android:layout_width">fill_parent</item>
25        <item name="android:layout_height">wrap_content</item>
26        <item name="android:textColor">#ffffff</item>
27        <item name="android:gravity">center</item>
28        <item name="android:layout_margin">1dp</item>
29        <item name="android:textSize">18sp</item>
30        <item name="android:textStyle">bold</item>
31        <item name="android:shadowColor">#000000</item>
32        <item name="android:shadowDx">1</item>
33        <item name="android:shadowDy">1</item>
34        <item name="android:shadowRadius">2</item>
35    </style>
36
37    <style name="BlackText">
38        <item name="android:layout_width">fill_parent</item>
39        <item name="android:layout_height">wrap_content</item>
40        <item name="android:textColor">#000000</item>
41        <item name="android:gravity">center</item>
42        <item name="android:layout_margin">5dp</item>
43        <item name="android:textSize">25sp</item>
44        <item name="android:textStyle">bold</item>
45        <item name="android:shadowColor">#ffffff</item>
46        <item name="android:shadowDx">1</item>
47        <item name="android:shadowDy">1</item>
48        <item name="android:shadowRadius">2</item>
49    </style>
50
51 </resources>

```




Step 5:

Understanding Android JSON Parsing!

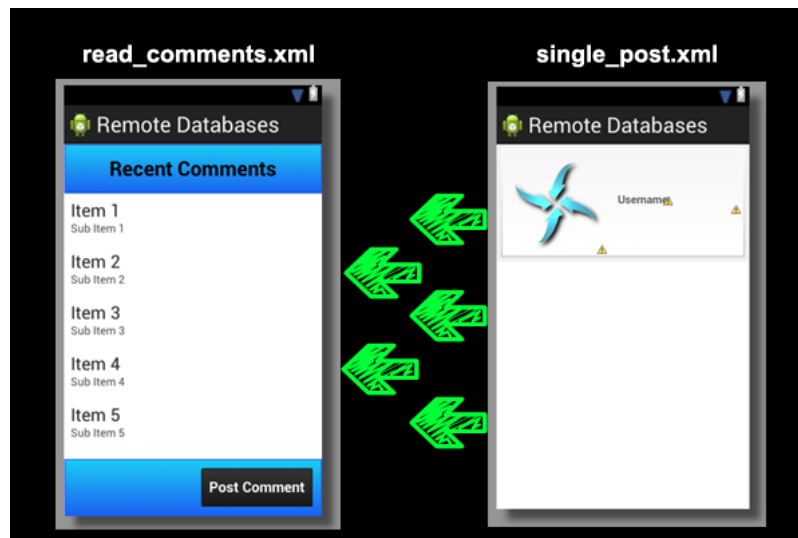
I apologize if it seems that I'm jumping all over the place in this tutorial, but trust me, we are almost done! We have setup a template of a single list item, and now we need to setup the xml for that will manage all of these individual list items. I called this XML file, `read_comments`, because this will be the layout where we read our comments from the JSON information.

`read_comments.xml`

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:background="#fff" >
6
7      <LinearLayout
8          android:id="@+id/top_layover"
9          android:layout_width="fill_parent"
10         android:layout_height="wrap_content"
11         android:layout_alignParentLeft="true"
12         android:layout_alignParentTop="true"
13         android:background="@drawable/blue_gradient"
14         android:orientation="horizontal" >
15
16         <TextView
17             style="@style/BlackText"
18             android:layout_height="wrap_content"
19             android:layout_gravity="center"
20             android:gravity="center"
21             android:text="@string/read_comments_title"
22             android:textAppearance="?android:attr/textAppearanceLarge" />
23     </LinearLayout>
24
25     <ListView
26         android:id="@android:id/list"
27         android:layout_width="fill_parent"
28         android:layout_height="wrap_content"
29         android:layout_above="@+id/bottom_layover"
30         android:layout_below="@+id/top_layover"
31         android:background="#fff"
32         android:divider="@android:color/transparent"
33         android:scrollbars="none" />
34
35     <LinearLayout
36         android:id="@+id/bottom_layover"
37         android:layout_width="fill_parent"
38         android:layout_height="wrap_content"
39         android:layout_alignParentBottom="true"
40         android:layout_alignParentLeft="true"
41         android:background="@drawable/blue_gradient"
42         android:orientation="horizontal"
43         android:weightSum="2" >
44
45         <LinearLayout
46             android:layout_width="0dp"
47             android:layout_height="wrap_content"
48             android:layout_weight="1"
49             android:orientation="vertical" >
50
51         <LinearLayout
52             android:layout_width="0dp"
53             android:layout_height="wrap_content"
54             android:layout_weight="1"
55             android:orientation="vertical" >
56
57             <Button
58                 android:id="@+id/post_comment"
59                 style="@style/WhiteText"
60                 android:layout_width="wrap_content"
61                 android:layout_height="wrap_content"
62                 android:layout_gravity="right"
63                 android:background="@drawable/black_button"
64                 android:onClick="addComment"
65                 android:text="@string/post_comment" />
66
67         </LinearLayout>
68     </LinearLayout>
69
70 </RelativeLayout>

```



Now, you can visualize what our application is going to look like. Our "read_comment.xml" will view all of our list items (single_post.xml) which will organize each of our JSON elements, such as title, message, username, etc.

Step 6: Finally, let's parse and display the JSON data!

Finally, the time has come where we will wrap up our project. We just need to modify the **ReadComments.java** file so that it will setup all of our list items, and display the list items within our ListView.

First, let's setup our class with all of the basics we are familiar with now. The only difference in this version of **ReadComments.java** is that our activity extends **ListActivity**, instead of just "Activity", and also that we will execute the "loading of comments" within the **onResume** method. We do this because if the user post a comment we want the list to be updated so that they can see there comment.

ReadComments.java v.01

```

1  package com.example.mysqltest;
2
3  import java.util.ArrayList;
4  import java.util.HashMap;
5  import java.util.Locale;
6
7  import org.json.JSONArray;
8  import org.json.JSONException;
9  import org.json.JSONObject;
10
11 import android.app.ListActivity;
12 import android.app.ProgressDialog;
13 import android.content.Intent;
14 import android.content.SharedPreferences;
15 import android.os.AsyncTask;
16 import android.os.Bundle;
17 import android.preference.PreferenceManager;
18 import android.util.Log;
19 import android.view.View;
20 import android.widget.AdapterView;
21 import android.widget.AdapterView.OnItemClickListener;
22 import android.widget.Button;
23 import android.widget.ImageView;
24 import android.widget.LinearLayout;
25 import android.widget.ListAdapter;
26 import android.widget.ListView;
27 import android.widget.SimpleAdapter;
28 import android.widget.TextView;
29
30 public class ReadComments extends ListActivity {
31
32     // Progress Dialog
33     private ProgressDialog pDialog;
34
35     //php read comments script
36
37     //localhost :
38     //testing on your device
39     //put your local ip instead, on windows, run CMD > ipconfig
40     //or in mac's terminal type ifconfig and look for the ip under en0 or en1
41     // private static final String READ_COMMENTS_URL =
42     "http://xxx.xxx.x.x:1234/webservice/comments.php";
43
44     //testing on Emulator:
45     private static final String READ_COMMENTS_URL =
46     "http://10.0.2.2:1234/webservice/comments.php";

```

```

46 //testing from a real server:
47 //private static final String READ_COMMENTS_URL =
"http://www.mybringback.com/webservice/comments.php";
48
49 //JSON IDS:
50 private static final String TAG_SUCCESS = "success";
51 private static final String TAG_TITLE = "title";
52 private static final String TAG_POSTS = "posts";
53 private static final String TAG_POST_ID = "post_id";
54 private static final String TAG_USERNAME = "username";
55 private static final String TAG_MESSAGE = "message";
56 //it's important to note that the message is both in the parent branch of
57 //our JSON tree that displays a "Post Available" or a "No Post Available"
message,
58 //and there is also a message for each individual post, listed under the "posts"
59 //category, that displays what the user typed as their message.
60
61
62 //An array of all of our comments
63 private JSONArray mComments = null;
64 //manages all of our comments in a list.
65 private ArrayList<HashMap<String, String>> mCommentList;
66
67 @Override
68 protected void onCreate(Bundle savedInstanceState) {
69     super.onCreate(savedInstanceState);
70     //note that use read_comments.xml instead of our single_post.xml
71     setContentView(R.layout.read_comments);
72 }
73
74 @Override
75 protected void onResume() {
76     // TODO Auto-generated method stub
77     super.onResume();
78     //loading the comments via AsyncTask
79     new LoadComments().execute();
80 }
81
82 public void addComment(View v)
83 {
84     Intent i = new Intent(ReadComments.this, AddComment.class);
85     startActivity(i);
86 }
87
88 /**
89  * Retrieves json data of comments
90  */
91 public void updateJSONdata() {
92 }
93
94
95 /**
96  * Inserts the parsed data into our listview
97  */
98 private void updateList() {
99 }
100
101
102 public class LoadComments extends AsyncTask<Void, Void, Boolean> {
103
104     @Override
105     protected void onPreExecute() {
106         super.onPreExecute();
107         pDialog = new ProgressDialog(ReadComments.this);
108         pDialog.setMessage("Loading Comments...");
109         pDialog.setIndeterminate(false);
110         pDialog.setCancelable(true);
111         pDialog.show();
112     }
113
114     @Override
115     protected Boolean doInBackground(Void... arg0) {
116         //we will develop this method in version 2
117         updateJSONdata();
118         return null;
119     }
120
121
122     @Override
123     protected void onPostExecute(Boolean result) {
124         super.onPostExecute(result);
125         pDialog.dismiss();
126         //we will develop this method in version 2
127         updateList();
128     }
129 }
130 }

```

This should look pretty familiar to most of the activities we have set up in this mini series. I just extracted what we will put in our doInBackground and onPostExecute methods into their own methods, updateJSONdata and

updateList. There are also two new variable types, JSONArray and a String Hashmapped ArrayList. I believe I have gone over ArrayList and ListActivities in my beginner Android course, but I'm sure you'll be able to figure out what's going on once we finish our list activity.

So, now every time we call onResume, we will need async task to update JSON data, and then update our list. Let's start with updating JSON data.

ReadComments.java

```

1  ...
2  /**
3   * Retrieves recent post data from the server.
4   */
5   public void updateJSONdata() {
6
7       // Instantiate the arraylist to contain all the JSON data.
8       // we are going to use a bunch of key-value pairs, referring
9       // to the json element name, and the content, for example,
10      // message it the tag, and "I'm awesome" as the content..
11
12      mCommentList = new ArrayList<HashMap<String, String>>();
13
14      // Bro, it's time to power up the J parser
15      JSONParser jParser = new JSONParser();
16      // Feed the beast our comments url, and it spits us
17      //back a JSON object. Boo-yeah Jerome.
18      JSONObject json = jParser.getJSONFromUrl(READ_COMMENTS_URL);
19
20      //when parsing JSON stuff, we should probably
21      //try to catch any exceptions:
22      try {
23
24          //I know I said we would check if "Posts were Avail." (success==1)
25          //before we tried to read the individual posts, but I lied...
26          //mComments will tell us how many "posts" or comments are
27          //available
28          mComments = json.getJSONArray(TAG_POSTS);
29
30          // looping through all posts according to the json object returned
31          for (int i = 0; i < mComments.length(); i++) {
32              JSONObject c = mComments.getJSONObject(i);
33
34              //gets the content of each tag
35              String title = c.getString(TAG_TITLE);
36              String content = c.getString(TAG_MESSAGE);
37              String username = c.getString(TAG_USERNAME);
38
39
40              // creating new HashMap
41              HashMap<String, String> map = new HashMap<String, String>();
42
43              map.put(TAG_TITLE, title);
44              map.put(TAG_MESSAGE, content);
45              map.put(TAG_USERNAME, username);
46
47              // adding HashList to ArrayList
48              mCommentList.add(map);
49
50              //annndddd, our JSON data is up to date same with our array list
51          }
52      } catch (JSONException e) {
53          e.printStackTrace();
54      }
55  }
56  ...
57  ...

```

I think I commented enough info to give you the gist of what is going on with that method. So, all we have left is to update the ListActivities' GUI List with the newly updated JSON data.

ReadComments.java

```

1  ...
2  /**
3   * Inserts the parsed data into the listview.
4   */
5   private void updateList() {
6       // For a ListActivity we need to set the List Adapter, and in order to do
7       //that, we need to create a ListAdapter. This SimpleAdapter,
8       //will utilize our updated Hashmapped ArrayList,
9       //use our single_post xml template for each item in our list,
10      //and place the appropriate info from the list to the
11      //correct GUI id. Order is important here.
12      ListAdapter adapter = new SimpleAdapter(this, mCommentList,
13          R.layout.single_post, new String[] { TAG_TITLE, TAG_MESSAGE,
14              TAG_USERNAME }, new int[] { R.id.title, R.id.message,
15              R.id.username });
16

```

```
17 // I shouldn't have to comment on this one:
18 setListAdapter(adapter);
19
20 // Optional: when the user clicks a list item we
21 //could do something. However, we will choose
22 //to do nothing...
23 ListView lv = getListView();
24 lv.setOnItemClickListener(new OnItemClickListener() {
25
26     @Override
27     public void onItemClick(AdapterView<?> parent, View view,
28         int position, long id) {
29
30         // This method is triggered if an item is click within our
31         // list. For our example we won't be using this, but
32         // it is useful to know in real life applications.
33
34     }
35 });
36 }
37 ...
```

That's all folks! Our application should be able to send and receive information from a remote server, parse JSON data, and look relatively stylish!

I hope you enjoyed this mini series, and I hope you've learned something new. If it did, I would love to hear from you on our twitter, facebook, or G+ page. Thanks again, and have a great devLife!

Download the Source!

[Download Source](#)

That's it!

[Support the Cause!](#)

Well, I hope that you have enjoyed this Intermediate Android Mini-Series on Android development with remote databases. Hopefully, you now understand how a PHP based web service, a remote MySQL database, JSON, and Android work together in order to create powerfully applications. Wishin' you the best in your future development projects!

-Trav

If you like what we're doing, please share it! We would also love to hear from you on facebook, twitter, g+, etc, etc, etc... laterz gatorz

[Previous Lesson](#)[Donate](#)

Author: trav

I'm just an average guy that love programming.

Share This Post On

Submit a Comment

Your email address will not be published. Required fields are marked *

Website



CAPTCHA Code *

Comment

You may use these HTML tags and attributes: <abbr title=""> <acronym title=""> <blockquote cite=""> <cite> <code> <del datetime=""> <i> <q cite=""> <strike>

Submit Comment

Tutorials

www.mybringback.com/tutorial-series/13239/android-mysql-php-json-part-6-json-parsing-and-android-design/

14/14